Structure in the Recurrence

Conditioning Genomic Foundation Model Pretraining on 3D Chromatin Contacts

Project technical record

Compiled August 13, 2026

No experimental results appear in this document. No model has been trained. The abstract is a proposal abstract. Every number reported comes from a data-processing, parameter-accounting or unit-test script in the repository, never from a training run. The working rule for this project is that no figure reaches a draft unless it exists first in a logged output file.

Abstract

Genomic foundation models are pretrained on DNA sequence alone, yet much of the regulatory logic they are asked to predict is carried by the genome's three-dimensional folding, which determines which enhancers can reach which promoters. Existing work combining sequence with chromatin contact data does so after the sequence representation is already fixed: CHROME trains an encoder on individual loci and only then adds a graph-attention layer over Hi-C contacts, and Evo2HiC distils a frozen Evo 2 into a compact encoder using Hi-C as a contrastive alignment target. In both, contact data shapes which learned features are combined, never which features are learned. We propose instead to place chromatin structure inside the pretraining computation. Contact-derived features modulate the state-transition dynamics of a bidirectional Mamba backbone, so Hi-C signal changes how far information propagates along the sequence rather than only what each position encodes. The mechanism adds 1.70% to parameter count over a matched sequence-only baseline. We evaluate whether a structure-shaped representation transfers—a question no sequence-to-structure model has been asked, as a forward-citation sweep over 318 papers citing Akita and Orca confirms—using long-range regulatory and variant-effect tasks together with a perturbation-sensitivity protocol on which Evo 2 has been reported to fail. Controls include structure shuffled at pretraining time and a distance-matched rewiring that tests whether any apparent structural signal is genomic distance in disguise.

Keywords: genomic foundation models; 3D genome organisation; Hi-C; topologically associating domains; state-space models; Mamba; self-supervised pretraining; CTCF; variant effect prediction; representation transfer.

Contents

1 The problem

An enhancer can sit 10^5 to 10^6 base pairs from the promoter it regulates. Nothing in the linear arrangement of the sequence explains how one reaches the other. The genome resolves this by folding: a ring-shaped cohesin complex extrudes a loop of DNA until it stalls at a pair of CTCF binding sites, and the resulting loop places the enhancer physically against the promoter.

The stalling condition matters for what follows. Cohesin halts only when the two CTCF motifs are *convergent*—pointing toward each other. Motif presence is insufficient; orientation decides. A sequence-only model sees the motif but receives no training signal that makes orientation pairing across a 500 kb gap consequential.

Three structures recur in the analysis. A *topologically associating domain* (TAD) is a region that contacts itself far more than its surroundings; enhancers act almost exclusively within their own TAD, so boundaries determine reachability. A *loop* is a point-to-point contact between two anchors. An *A/B compartment* is a coarse partition into open, active and closed, inactive regions.

2 Related work and the gap

Work	Seq.	Struct.	Structure enters at	Needed at inference
HiCFoundation (2026)	no	yes	pretraining	yes
Hi-Cformer (2025)	no	yes	pretraining	yes
Evo 2 (2026)	yes	**no**	—	no
Caduceus (2024)	yes	**no**	—	no
GraphReg (2022)	feat.	yes	supervised, downstream	yes
CHROME (2026)	yes	yes	**stage 2, post-hoc**	**yes**
Akita / Orca	yes	target	supervised target	no
Evo2HiC (2025)	yes	yes	contrastive distillation	no
This project	**yes**	**yes**	**self-supervised pretraining**	**target: no**

Table 1: Where 3D structure first influences the model. The bottom row is unoccupied in the current literature.

2.1 The two nearest competitors

CHROME (Ye et al., *Briefings in Bioinformatics* 27(4):bbag360, 2026) filters Hi-C against a self-avoiding polymer null model simulated over 5×10^5 chains, retaining 1.8–6% of contacts, then runs two graph-attention layers over signal-centred subgraphs with a 4 Mb receptive field at 5 kb bins. Training has two stages: the encoder is trained on centre nodes alone until validation saturates, and only afterwards are early layers frozen and the graph module attached. Supervision is 751 ENCODE assays. Structure never influences what the sequence encoder learns, and the contact graph is required at inference.

Evo2HiC (Fang et al., bioRxiv 10.1101/2025.11.18.689171) distils a frozen Evo 2 (7B) into a 3.6M-parameter CNN using two SigLIP contrastive objectives, the second aligning the student to Hi-C patch embeddings. Structure genuinely shapes a sequence encoder, but by aligning output embeddings, with no objective over nucleotides. A keyword count over the full preprint returns zero occurrences of *ClinVar*, *pathogenic*, *eQTL*, *variant*, *masked*, *MLM*, *Mamba* and *state space*.

Their third stated limitation is that they froze Evo 2 because fine-tuning it is computationally hard, and that they intend to develop strategies for adapting it with Hi-C data—this project's question, named as work they have not done.

2.2 Two external findings that shaped the plan

Lee (arXiv:2604.07196, April 2026) probed Evo 2 (7B) on 231 TAD boundary regions and 120 convergent CTCF loops. Boundary deletions were penalised *less* than GC- and size-matched random controls (paired Wilcoxon $p = 0.405$); CTCF inversions and deletions likewise ($p = 0.021$, $p = 0.006$, in the wrong direction). Generated loops reached median enrichment 0.054 against a reference of 0.388. The paper concludes Evo 2 "has learned local CTCF grammar but misses higher-order 3D organization" and prescribes bidirectional architectures with explicit 3D contact inputs, citing Caduceus. This substantially defuses failure mode F3 (Section 6): if a 7B-parameter model with a 1M-token context cannot infer TAD-scale structure from sequence, a 7.7M-parameter model will not either.

DNALongBench (*Nature Communications* 16(1):10108, 2025) provides five long-range tasks including enhancer–target interaction, eQTL and contact-map prediction, already benchmarking Caduceus-Ph and Akita. Adopting it for evaluation removes the objection that tasks were chosen to flatter. Calibration: contact-map prediction is the hardest task in the suite, best reported correlation 0.233, by Akita.

2.3 The negative result the contribution rests on

A forward-citation sweep over 318 papers—118 citing Orca, 200 citing Akita—found no work that takes either model's learned representations and evaluates them on a task other than predicting folding. Their variant applications all run the model twice, on reference and mutant sequence, and compare predicted contact maps. Internal embeddings are never extracted, frozen and probed, or fine-tuned into another head.

3 Method

3.1 Baseline recurrence

Per layer and direction, with channel index i and state index n:

$$(z, v) = \mathrm{split}(W_{\mathrm{in}} u_t) \tag{1}$$
$$x_t = \mathrm{SiLU}(\mathrm{Conv1d}(v)_t) \tag{2}$$
$$(\delta'_t, B_t, C_t) = \mathrm{split}(W_x x_t) \tag{3}$$
$$\Delta_t = \mathrm{softplus}(W_\delta \delta'_t + b_\delta) > 0 \tag{4}$$
$$A_{i,n} = -\exp(A^{\log}_{i,n}) < 0 \tag{5}$$
$$\bar{A}_{t,i,n} = \exp(\Delta_{t,i} A_{i,n}) \in (0, 1) \tag{6}$$
$$h_{t,i,n} = \bar{A}_{t,i,n}\, h_{t-1,i,n} + \Delta_{t,i} B_{t,n} x_{t,i} \tag{7}$$
$$y_{t,i} = \textstyle\sum_n C_{t,n} h_{t,i,n} + D_i x_{t,i} \tag{8}$$

3.2 The structural modification

Let $s_t \in \mathbb{R}^{d_s}$ be the structural context at position t, derived from Hi-C. Two terms are added:

$$\Delta_t = \mathrm{softplus}\big(W_\delta \delta'_t + b_\delta + W_{\Delta s} s_t\big) \tag{9}$$
$$p_t = \mathrm{softplus}\big(w_g^\top s_t + b_g\big) \;\geq 0 \tag{10}$$
$$\bar{A}_{t,i,n} = \exp\big(\Delta_{t,i} A_{i,n} - p_t\big) \tag{11}$$

The first retunes the memory length of each channel. The second is a non-negative damping that only shortens memory: at a TAD boundary the model can raise it and cut the state.

Why this is not an input feature. Appending s_t to the input would reach only $\bar{B}_t x_t$, the term governing how much of the current token enters. It cannot alter $\bar{A}_t$, which governs what survives. Because Δ enters $\bar{A}$ inside an exponential, structure here sets the effective memory horizon

$$\tau_{t,i} = \frac{-1}{\Delta_{t,i} A_{i,n} - p_t} \quad \text{(tokens)}, \tag{12}$$

that is, how *far* information travels rather than what is encoded. This is the class of constraint that contrastive alignment on output embeddings cannot express, and it is the entire novelty claim.

Relationship to hard state resets. As $p_t \to \infty$, $\bar{A}_t \to 0$ and $h_t = \bar{B}_t x_t$: a hard reset. The mechanism is the soft, learned relaxation of TAD-conditioned state resetting, which is recorded as future work.

Initialisation. $W_{\Delta s} = 0$, $w_g = 0$, $b_g = -4$. The model is then numerically indistinguishable from the baseline at step zero, so any divergence is attributable to learned structural signal. Zero initialisation of $W_{\Delta s}$ does not block learning, since $\partial L / \partial W_{\Delta s} = (\partial L / \partial \Delta)\, s_t^\top$.

3.3 Parameter accounting

Computed by instantiating both models and summing parameter tensors (`scripts/param_accounting.py`, `scripts/test_model.py`).

Configuration	Total	Added	Δ
Sequence-only baseline	7,725,312	—	—
$d_s = 2$, with encoder	7,758,354	33,042	+0.428%
$d_s = 8$, with encoder	7,856,952	131,640	+1.704%
$d_s = 8$, **no encoder** (recommended)	7,856,672	131,360	+1.700%

Table 2: All configurations sit inside the 5% matched-parameter budget.

4 Implementation

4.1 Scan backends

The selective scan has two implementations, selected by device.

The reference implementation is a sequential loop over positions: correct, differentiable, and what the CPU unit tests run at small L. It is not viable at $L = 32{,}768$, which would require $32{,}768 \times 16 \times 2$ Python iterations per forward pass.

Production requires a fused kernel, and the two structural terms differ sharply in how they cooperate with the stock one. **The Δ bias is free**: `mamba_ssm`'s `selective_scan_fn` takes `delta` as an argument, so the bias is added before the call. **The permeability term p_t is not expressible** through that interface: it needs $\bar{A} = \exp(\Delta A - p)$, and folding p into Δ would require $\Delta' = \Delta + \log(g)/A_n$, which depends on the state index n. A single per-channel Δ cannot produce a decay uniform across n.

A custom Triton kernel was therefore written (`src/chromfm/scan_triton.py`), carrying p through both forward and backward. It uses one program per (batch, channel) pair, checkpoints

the state at chunk boundaries, and in the backward replays each chunk from its checkpoint before running the adjoint recurrence, keeping memory at $O(BD\,\frac{L}{\text{chunk}}N)$ rather than $O(BDLN)$.

The Triton kernel has never been executed. It was written on a CPU-only machine with no CUDA device and no Triton install. The recurrence and its adjoint were derived by hand and the code is syntactically complete, but nothing is numerically confirmed. `scripts/validate_kernel.py` must be run on the GPU box before any training. A scan with a subtly wrong gradient produces a smooth, plausible loss curve while training the wrong model.

Both arms must use the same backend, or the matched-compute comparison is confounded. The baseline therefore runs on this scan too, even though it never supplies p.

4.2 Dataset layer

Window		32,768 bp
Train stride		16,384 (50% overlap)
Evaluation stride		32,768 (no overlap)
Train windows		5,422
Validation windows	273	(chr9:120,029,184–128,974,848)
Test windows	242	(chr9:130,023,424–137,953,280)
Dropped	1,054 on N fraction, 799 on structural coverage, 126 buffer	

Table 3: Window index. Verified: zero training windows touch either held-out region; both evaluation splits are fully contained with no internal overlap.

Window length follows from the memory-horizon measurement rather than convention: the 16-layer relayed horizon is roughly 16,000 tokens, so 32,768 already exceeds what the stack can span.

Three policies are implemented in the dataset builder rather than in either training script, so the two arms cannot diverge on them:

(i) ϕ **is interpolated, not stepped.** Linear interpolation between bin centres. Measured inside one window, a step function yields 7 distinct values per feature; interpolation yields 32,757. Seven values across a window is the precondition for failure mode F4.

(ii) **N has its own token and a budget.** 12.00% of chr9 is unresolved; windows above 10% N are dropped.

(iii) **Invalid bins are masked, never NaN.** A position is structurally valid only when both flanking bins are usable. Invalid positions receive $\phi = 0$, the standardised mean, plus a validity flag. A NaN reaching $W_{\Delta s}s_t$ would poison the entire recurrence.

Reverse-complement augmentation reverses and complements the tokens, then reverses ϕ and negates its two antisymmetric coordinates (`directionality_2Mb`, `upstream_mass_frac`).

5 Pilot data

5.1 Provenance

Source is 4D Nucleome experiment set **4DNES3JX38V5**: in situ Hi-C on GM12878, MboI with bio-dATP, Lieberman Aiden lab, published as Rao et al. 2014 (PMID 25497547), aligned to GRCh38. The multi-resolution contact matrix (4DNFIXP4QG5B) is 27.4 GB and is never downloaded; the pipeline reads a near-diagonal band directly over HTTP range requests in six and a half minutes. Boundary calls, insulation and compartment tracks are mirrored as independent validation targets, not as model inputs. Sequence is Ensembl release 113; annotation is GENCODE release 47.

5.2 Measured properties

chr9 length, from the contact matrix	138,394,717 bp
chr9 length, from the Ensembl FASTA	138,394,717 bp
Bins at 5 kb	27,679
Band entries retained	7,108,483
Band occupancy	0.6451
Bins with no balancing weight	19.31%
Unresolved sequence (N)	12.00%
Bins with a complete feature vector	21,519 (77.74%)

The two identical lengths are the first coordinate-alignment check. A second, stronger one emerged unplanned: the pipeline returned nothing for chr9 between 48 and 58 Mb. That is the centromere and the 9q12 heterochromatic block, which GRCh38 represents as a gap. The sequence file is 100% unresolved from approximately 46 to 60 Mb, and the contact matrix has no usable bins from 43.27 to 60.52 Mb. Two files from different institutions placing the same feature at the same coordinates is stronger evidence than a matching chromosome length.

5.3 Structural features

Index	Feature	Window	Under reverse complement
0–2	insulation score	100, 250, 500 kb	symmetric
3	directionality index	2 Mb	**antisymmetric**
4	log contact density	2 Mb	symmetric
5	upstream mass fraction	2 Mb	**antisymmetric**
6	short-to-long range ratio	100 kb / 2 Mb	symmetric
7	A/B compartment eigenvector	250 kb, GC-oriented	symmetric

Table 4: The symmetry column is load-bearing: failure mode F7 is the case where features 3 and 5 cancel between the forward and reverse passes.

5.4 Validation against independent tracks

Our feature	Against 4DN's own track	Pearson r
Insulation, 100 kb window	4DNFIBMOGOZC	+0.9969
Insulation, 250 kb window	4DNFIBMOGOZC	+0.7802
Insulation, 500 kb window	4DNFIBMOGOZC	+0.4895
A/B compartment eigenvector	4DNFILYQ1PAY	+0.9759

Table 5: The 100 kb window reproduces 4DN's track almost exactly, identifying their diamond window and confirming the implementation. Declining correlation at larger windows is the expected consequence of comparing different window sizes against a fixed reference.

A TAD and a loop are both visible in the processed data. The loop lies at chr9:136,485,000 paired with 136,625,000, 140 kb apart, at 7.5 times its distance-matched expectation, with six CTCF ChIA-PET read pairs from an independent assay at the same anchors. Its upstream anchor falls on *NOTCH1* (chr9:136,494,433–136,546,048).

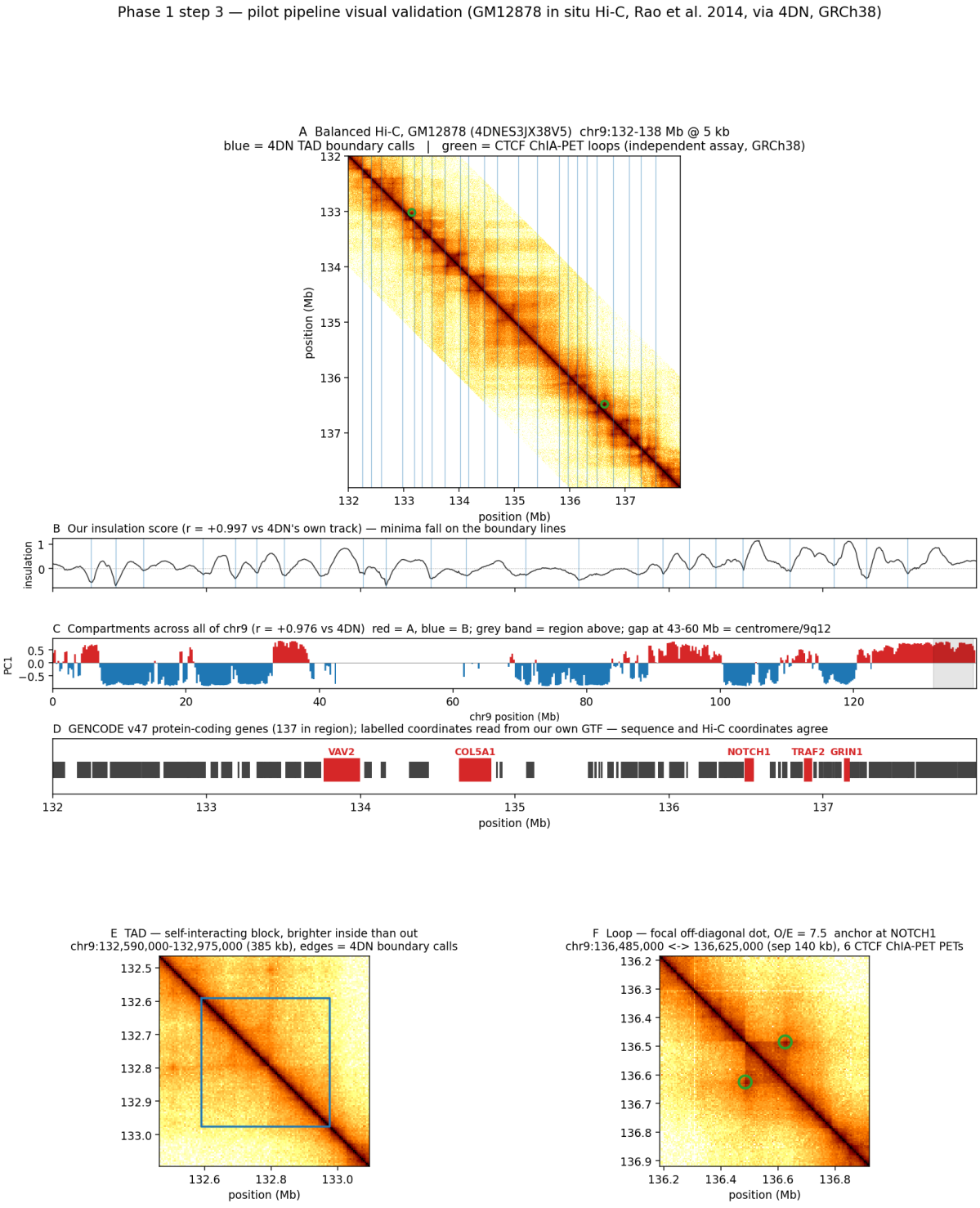

Figure 1: Pilot pipeline validation. (A) Balanced contacts with 4DN boundary calls in blue and CTCF ChIA-PET loops in green. (B) Our insulation score. (C) Compartments across all of chr9; the blank interval at 43–60 Mb is the centromere. (D) GENCODE genes with coordinates read from our own annotation. (E) A 385 kb TAD. (F) The NOTCH1-anchored loop.

6 Failure modes and controls

6.1 Shuffled-structure controls

ID	Control	Destroys	Rules out
S0	zero the signal	everything	establishes the sequence-only floor at identical parameter count
S1	permute across bins	alignment and autocorrelation	primary reliance probe
S2	circular shift, 10 Mb	alignment only	"any smooth auxiliary channel would do"
S3	distance-matched rewire	locus-specific structure	**"structure is genomic distance in disguise"**

Table 6: S3 is the control that can end the project.

Contact probability falls smoothly with genomic distance. If a model performs as well on distance-matched fake structure as on real structure, what it learned is a re-parameterised positional prior and the central claim is false however good the headline number looks.

The gate passes only if degradation under shuffling is positive with a 95% bootstrap confidence interval excluding zero *and* is at least twice the across-seed standard deviation of the real-structure runs. Significance alone is insufficient; the effect-size floor prevents a p-value standing in for an effect.

6.2 The eight failure modes

	Failure	Symptom	Status
F1	bias absorbed into b_δ	trains to baseline loss	highest probability; mitigated by genome-wide standardisation
F2	softplus saturation	structural weights never move	**observed**; b_g changed $-8 \to -4$
F3	structure inferable from sequence	structural input redundant	**largely defused** by Lee's Evo 2 probe
F4	bin longer than memory horizon	trains to baseline loss	**precondition confirmed**; mitigated by interpolation
F5	permeability gate stays at zero	half the mechanism inert	monitor the distribution of p
F6	gradient starvation	structural pathway never learns	1.7% of parameters behind two nonlinearities
F7	forward and reverse cancel	trains to baseline loss	two of eight features flip sign under RC
F8	structural encoder collapses	reduces to F1	**eliminated** if the encoder is removed

F1, F4 and F7 produce the same symptom: the model trains to baseline performance and raises no error. Four diagnostics therefore run continuously—the variance the structural pathway contributes to the timescale, the distribution of the permeability penalty, the growth of the

structural weight norm, and a probe asking whether the baseline model's hidden states already predict the structural signal.

6.3 Memory horizon, measured

At the reference initialisation, horizons run from 0.6 tokens at the fast end to 994 at the slowest, median 14.9. A 5 kb bin is 5,000 tokens, so no channel-state pair spans a single bin; $\tau_{\max}/\text{bin} = 0.199$. Sixteen layers relaying extends reach to roughly 15,900 tokens, still short of a 385 kb TAD.

Δ is learned and can grow, so this is not proof of failure. It constrains the sequence-only baseline exactly as much as the structural model, which raises a question worth settling before spending GPU time: can a model of this size represent TAD-scale dependencies at all? **Phase 3 must therefore report the trained memory horizon, and that measurement gates Phase 4.**

7 Decisions of record

	Decision	Choice	Reasoning
1	Mechanism	structural bias in the recurrence	unreachable by contrastive alignment on output embeddings; contains hard state resets as its limit; cheap to falsify
2	Reverse complement	Caduceus-PH (augmentation)	equivariance is orthogonal to the hypothesis; coupling them makes a null result uninterpretable
3	Hi-C resolution	5 kb, 1 kb as ablation	comparability with CHROME; both levels are in the same file
4	Cross-species scope	out for this paper	Evo2HiC uses 177 species, HiC-Foundation 316; entering that lane splits compute against better-resourced groups
5	Permeability term	**keep**, via custom kernel	retains the soft-reset semantics and the link to future work

8 Change log

Every substantive change to the project is recorded here with its reason.

2026-08-12 — F4 resolved: the cap was `dt_min`, and it was exact

- **Cause, in one line of code.** `model.py::_init_dt` draws Δ log-uniform over $[\texttt{dt\_min}, \texttt{dt\_max}] = [10^{-3}, 10^{-1}]$, and `model.py` line 128 sets $A = -\text{arange}(1, \texttt{d\_state} + 1)$ so $|A| \geq 1$. Since $\tau = 1/(\Delta|A|)$, $\tau_{\max}$ **at initialisation is exactly** $1/\texttt{dt\_min} = 1{,}000$ **tokens**. Phase 3 measured 999.5. Mamba's reference `dt_min` was a hard ceiling on the memory horizon 100× below TAD scale, and training never moved the bulk: median τ 14.6 at init, 14.2 after 2000 steps. The architecture could not express what the mechanism conditions on.
- **Change:** `dt_min` $10^{-3} \to 10^{-6}$, `dt_floor` $10^{-4} \to 10^{-7}$, `dt_max` unchanged. Promoted from `_init_dt` defaults to `ModelConfig` fields so `train.py` records them in `run_config.yaml`: a

run whose memory horizon is missing from its own provenance cannot be compared with another.

- **Measured at init, not asserted** (`f4_memory_horizon.py`, corroborated by `tau_stats()` on the instantiated model): median τ 14.9 → 472.5; p90 108.8 → 47,940; p99 497.4 → 290,487; $\tau_{\max}$ 994 → 985,872. Fraction at TAD scale 0.0000 → 0.0433, which is 433× the 10^{-4} gate floor. The 385 kb validated TAD now fits inside $\tau_{\max}$ with 2.5× headroom.
- **Parameters unchanged at 7,725,312.** This is an initialisation change, not a capacity change, so the matched-compute comparison with the structural arm survives intact, as does structural/baseline init equivalence (`test_model.py`, max gap still exactly 0.0). 17/17 checks pass.
- **Why this lever and not the obvious ones.** Raising `d_state` adds states but $|A|$ still starts at 1, so the ceiling does not move. Raising `d_model` draws more channels from the same Δ range and breaks the parameter match. Widening the Δ range is the only lever that moves the ceiling at fixed capacity. Re-initialising A log-spaced over $[0.01, 16]$ is an independent lever that also works (measured $\tau_{\max}$ 99,432 on its own) and was deliberately *not* applied — one variable at a time, so a Phase 4 result can be attributed to something.
- **The `dt_floor` clamp is a trap, and it was nearly walked into.** `_init_dt` clamps Δ at `dt_floor`, capping τ at $1/$`dt_floor` whatever `dt_min` says. Lowering `dt_min` to 10^{-6} while leaving `dt_floor` at 10^{-4} gives $\tau_{\max}$ of exactly 10,000 tokens and reads as "the change did nothing". `_init_dt` now raises `ValueError` instead of capping silently, and `test_model.py` asserts $\tau_{\max} \geq 385{,}000$.
- **This invalidates the Phase 3 baseline for comparison purposes.** The three completed seeds were trained at `dt_min`$= 10^{-3}$ and are a different architecture. They stand as the record of *why* this change was made, but they are not the Phase 4 baseline and their $\sigma_{\text{real}} = 0.0040$ does not carry over. Phase 3 must be re-run on the new initialisation, and the F4 gate re-read from *trained* τ. The init numbers establish that TAD scale is now expressible, not that training preserves it — and Phase 3's central finding was exactly that init τ and trained τ diverge.

2026-08-12 — PHASE 3 COMPLETE: the F4 gate fails, Phase 4 must be re-scoped

Three seeds × 2000 steps of `BiMambaLM(structural=False)` on the chr9 pilot, all `COMPLETED`. Every number below is read by `scripts/phase3_report.py` out of `results/baselines/*/metrics.json` and `run_config.yaml`; the report is committed at `results/baselines/phase3_report.txt`.

- **The baseline trains, and the three seeds agree.** Final validation 1.5210 bits/nucleotide, sd 0.0040 across seeds (1.5164 / 1.5239 / 1.5227), accuracy 0.5240 ± 0.0014 — 0.4790 bits below the 2.000-bit uniform floor. **That sd 0.0040 is σ_{real}**, the quantity §4.1.3 needs for Phase 4's 2σ effect-size gate; it is now measured rather than assumed.
- **The F4 gate FAILS. Trained τ does not approach TAD scale.** Median τ 14.2 ± 0.2 tokens, p99 574.8 ± 55.3 — essentially unmoved from the 14.6 median measured at initialisation on 2026-08-07. The distribution's bulk never left the degenerate regime.
- **The tail is wildly seed-dependent and that is itself the finding.** $\tau_{\max}$ is 46,319 ± 61,333 tokens: a standard deviation larger than the mean. Per seed it is 10,584 / 117,139 / 11,235 — seed 1 alone crossed 100 kb, and only in `layer15.rev`, one of 32 layer-directions. Averaged over seeds, 0.3 of 32 layer-directions contain any triple at TAD scale. A single-seed run would have given either "comfortably short" or "crosses TAD scale" with equal ease. This is exactly the case the three-seed requirement exists for.
- **Gate arithmetic, against the criterion fixed earlier today:** mean $\tau_{\max}$ 46,319 (needs 100,000) — fail; mean relay heuristic 77,655 ± 51,512 (needs 100,000) — fail; exact mass

at TAD scale $6.954 \times 10^{-8} \pm 1.204 \times 10^{-7}$ (needs 10^{-4}) — fail by ∼1400×. All three arms fail independently, so the verdict does not rest on the threshold decision alone.

- **Action, per the §4.1.4 decision table: re-scope to sub-TAD structural signal, or raise `d_model`/`d_state`, before spending Phase 4 compute.** **This is a statement about the backbone, not about the structural mechanism**, and it constrains the baseline and the structural arm identically: a model that cannot carry information 100 kb cannot be helped by being told where the 100 kb boundaries are. It does not say structure is useless—φ carries sub-TAD signal too, and the interpolated s_t varies at every position—it says the TAD-scale framing of the claim is not supportable at this model size.
- **Why `tau_stats()` alone would have misled.** Bias-only $\tau_{\max}$ after training is 1,050.7 ± 21.0 tokens, against an empirical 46,319. The runbook asked for `tau_stats()`; had the gate been read from it, the answer would have been off by a factor of 44, because once trained, Δ is dominated by the input-dependent term $W_\Delta \delta'_t$ rather than by the bias.
- **Cost.** Six idle-culler kills across two days. No computation was ever lost—every resume came off a checkpoint—but the last of them stalled progress entirely once the cull interval fell below the 200-step checkpoint interval, and `--ckpt-every` was cut to 120 (PI's choice) to fit inside it. Seed wall-clock figures in the report are per final attempt and understate the interrupted seeds.

2026-08-12 — The F4 gate was unmeasurable as written, and is now pinned

- **The gate's mass test could not see the effect it tested for.** `empirical_tau` computes the summary's `tau_max` as an exact maximum over the full τ tensors (`train.py:362`) but its `frac_ge_*` from a random subsample of 20,000 per layer-direction (`train.py:352`). Different populations, so the two can disagree, and did: seed 1 at step 1000 logged `tau_max` 103,025 beside `frac_ge_100k` of exactly `0.0`. That is a detection floor, not a contradiction—the subsample cannot resolve anything rarer than $1/20{,}000 = 5 \times 10^{-5}$, and the true value was 5.96×10^{-8}, some 840× finer. Read from that field, the gate would have answered "no mass at TAD scale" by construction rather than by measurement, for any model in this regime. `phase3_report.py` now reads the exact per-layer fractions, which are computed on the full tensors and were already logged; no re-run was needed. `train.py` was deliberately *not* edited—it was mid-sweep, and changing it would have measured seed 2 with different code than seeds 0 and 1.
- **The gate's threshold was qualitative and is now operational (PI decision).** §4.1.4 said "τ reaches $\gtrsim$100 kb"; the data made the ambiguity load-bearing. The gate now passes iff mean $\tau_{\max} \geq 100{,}000$ tokens, or mean relay heuristic $\geq 100{,}000$ *and* exact `frac_ge_100k` $\geq 10^{-4}$. The mass floor is the new part. The prior condition was "any nonzero mass", which seed 1 met at 1.19×10^{-7}—about four triples per million, in 1 of 32 layer-directions, while median τ was 14.4 tokens and the other 31 layer-directions held nothing at TAD scale.
- **This mattered numerically, not just in principle.** On the data logged at the time of the decision, the mean relay heuristic stood at 98,636 tokens, 1.4% under its threshold, and it is the fastest-moving term in the gate (23,844 → 151,021 across seed 1). Under the old rule seed 2 could have carried it over the line and flipped the verdict to "proceed to Phase 4" on the strength of a quantity §4.1.4 itself calls "a HEURISTIC and not a bound". Under the new rule the mass floor binds and misses by roughly 1,700×.
- **Fixed before the evidence was in.** The threshold was set while seed 1 was at step 1200 of 2000 and seed 2 had not started, and before any verdict was computed. Recorded explicitly because a gate loosened or tightened after seeing its own answer is not a gate; the ordering is the whole point.

2026-08-11 — Phase 3 run supervision

The Phase 3 baseline is three seeds × 2000 steps at a measured 5.30 s/step, i.e. roughly 2.9 h per seed and 8.8 h in total. That exceeds the lifetime of an interactive session on this cluster, so the runs were made independent of one. Training results are recorded in a later entry.

- **The first launch was killed by session teardown** at step 200 of seed 0. Background processes started from an agent session die with it. Runs are now started with `setsid nohup`, giving them their own session, no controlling terminal and `init` as parent. Confirmed independently that this box has `KillUserProcesses=no` and no batch scheduler, so nothing reaps user processes at logout. **This was necessary but not sufficient, and the conclusion drawn from it at the time was wrong**—see the second entry for 2026-08-11, where the run was killed anyway by a mechanism none of these checks covered.
- `scripts/run_phase3.sh` **added** as the supervisor. It lives in the repository rather than a temporary directory—bash reads a script incrementally as it executes, so a driver under a path that is cleaned up mid-run can fail partway through the seed loop. It retries a failed seed up to 20 times with `--resume`, and decides completion from `status: COMPLETED` in the run's own `run_config.yaml`, which rank 0 writes only after the training loop exits normally. Deciding from an exit code instead would let a half-finished run be recorded as finished. A `flock` on fd 9 makes a second instance impossible: two supervisors would have two training processes writing the same `checkpoint.pt`.
- `scripts/phase3_guard.sh` **added** to restart the supervisor if it is killed outright. There is no writable user crontab and no `systemd --user` on this box, so it is a plain detached poller and does *not* survive a machine reboot; the entry point after a reboot is a single command, documented in the script's header.
- **Liveness must not be probed with `pgrep -f`.** The guard's first version used `pgrep -u $USER -f run_phase3.sh`, which matches any command line that merely mentions the script—an editor, a grep, the very command that launched the guard. It returned a false positive on its first poll and left training stopped. Liveness is now read from a pidfile and confirmed against that pid's `/proc/<pid>/cmdline`.
- `build_project_doc.py` **was silently failing its own rule check.** The script already returns exit 1 when `project-docs/` holds anything besides `project.tex` and `project.pdf`, but JupyterLab recreates a `.ipynb_checkpoints/` directory there within minutes of any build, because it holds the `.tex` open. The exit code had been read through a pipe to `tail`, which reports the exit status of `tail` rather than of the script, so the violation went unnoticed across two builds. The cleanup loop now removes that directory, and the exit code is read directly. It contained only Jupyter's autosave copies of files the script rebuilds, so nothing unique was discarded—verified before the first deletion by diffing the checkpoint against the live `.tex`: zero lines present only in the checkpoint.
- **The console log lags.** `train.py`'s output is piped through `grep -v "Can't initialize NVML"` inside the supervisor, and grep block-buffers when its stdout is a file, so training lines can appear tens of lines late. The supervisor's own markers bypass the pipe and are prompt. `scripts/phase3_progress.py` was added to read progress from `metrics.json`, which `train.py` writes atomically at every eval and which never lags. Adding `--line-buffered` to the supervisor's grep is deferred until the runs finish, because editing a script that bash is currently executing can corrupt its parse.
- **Checkpoint durability confirmed by reading the code:** `save_ckpt` writes to `.tmp` and then `Path.replace`, which is `os.replace` and atomic within a filesystem, so a kill during a write cannot leave a corrupt checkpoint. `metrics.json` is written the same way. At `--ckpt-every 200` the worst case is under 18 minutes of lost work.

- **Recovery verified under a real `SIGKILL`, not by inspection.** The trainer was killed outright at step 247 of seed 0 (accidentally, by a `pgrep -f` that matched the killing shell's own command line—the same false-match class as the guard bug above, and the reason the guard no longer uses `pgrep`). The chain behaved as designed: the step-200 checkpoint survived, the supervisor recorded `ProcessExitedException ... signal SIGKILL` and `attempt 1 exit=1`, declined to mark the run complete, and started attempt 2 sixty seconds later. Resume was then confirmed two ways: the step-200 record was still present in `metrics.json` with step 400 *appended* to it, where a restart from scratch would have reset the file to a single fresh row; and attempt 2 reached the step-400 eval 18.2 minutes after starting, which is 200 steps at 5.30 s/step, not the 400 steps a from-scratch run would have needed. Worst-case loss from an interruption is therefore one checkpoint interval, under 18 minutes, with no manual intervention.
- **Resume is not identical to an uninterrupted run.** Model, optimizer, scheduler, step count and RNG state are restored; the dataloader's position within the epoch is not. An interrupted seed therefore sees a different window order than an uninterrupted one. Recorded here so that a seed-to-seed discrepancy is not later mistaken for a real effect.
- **The 8.8 h was not shortened, deliberately.** Cutting 2000 steps to 1000 would halve the wall clock at no engineering cost, but the phase exists to measure *trained* τ for the F4 gate, and τ grows during training. Undertraining biases τ downward and would manufacture a false "TAD scale is not expressible, re-scope Phase 4" verdict. Confirmed with the PI that there is no deadline requiring the trade.
- **No further speedup is available cheaply.** The measured levers are exhausted: `num_warps=1` (1.42×, previous entry), two GPUs at 2.00× scaling, and a batch size already at saturation (batch 4 OOMs, and batch 8 with recompute is 13% worse per window than batch 2 without). Running the three seeds concurrently on one GPU each gains nothing, since DDP already scales at exactly 2× and the total work is fixed. The remaining real lever is a chunked parallel-prefix rewrite of the scan, which is currently a strictly sequential 32,768-iteration chain; it is deferred until after the F4 gate, since the gate may re-scope Phase 4 altogether.

2026-08-11 (second entry) — The idle culler, and why unattended runs are impossible on this box

The supervision built in the previous entry was tested against process death and survived it. It was never tested against the thing that actually kills runs here, and the run was lost a second time as a result.

- **What happened.** Seed 0 was killed at step 1000 while unattended. The console log ends in bash's `Terminated`, i.e. `SIGTERM`; the machine had not rebooted (uptime 126 days); the last checkpoint was written at 04:53:15 UTC, about ten minutes after the browser was closed. Both GPUs were free at 07:16 and the guard, supervisor and trainer were all gone—the entire chain, not one link of it.
- **Cause, confirmed by direct observation.** The hub runs `jupyterhub_idle_culler --timeout=600 --cull-every=60 --concurrency=5 --max-age=0`, and the whole user environment is a single systemd cgroup: `/proc/self/cgroup` reads `0::/system.slice/jupyter-238w1a5447.serv` Ten minutes after the last browser activity the hub stops that service, and systemd kills every process in the cgroup. **`setsid`, `nohup` and `PPID=1` give no protection against this.** They defend against `SIGHUP` and against a dying parent; a cgroup-wide kill is neither. The previous entry verified session independence and concluded the run was safe, which did not follow—cgroup membership was never checked.
- **Cost of the incident: wall clock only.** Restarted 07:17:00 from the step-1000 checkpoint. The step-1200 eval appended to the existing records at `val 1.5388` bits, continuing

down from step 1000's `1.5671` rather than restarting near the untrained `4.1004`. Resume works; roughly 2.4 h of idle wall clock was lost and no computed result was.

- **Consequence for planning: a 9-hour job cannot run unattended here.** No amount of process supervision fixes a 10-minute idle timeout, because the supervisor is inside the cgroup being killed. The options are to ask the administrators to raise the timeout or provide a batch queue, to keep a browser tab connected, or to accept periodic culling. A heartbeat that pokes the Jupyter API would defeat a resource policy the administrators set deliberately and is not being used.
- `scripts/phase3_status.sh` **added**—one read-only command reporting whether the chain is alive, every eval logged so far, per-seed completion, and GPU memory through the CUDA API (`nvidia-smi` is broken on this box). When nothing is running it prints the command to restart.
- `scripts/phase3_autostart.sh` **added**—idempotent restart. It exits immediately if a guard is already running or if all seeds are done, and otherwise starts the chain, which resumes the current seed from its last checkpoint. Liveness is read from a pidfile confirmed against `/proc/<pid>/cmdline`, not `pgrep -f`, for the reason in the previous entry.
- `scripts/jupyter_server_config.py` **added as an inert template, *not* installed.** Copied to `~/.jupyter/jupyter_server_config.py` it would run the autostart script at every single-user server start, so that reconnecting is itself the restart: `jupyterhub-singleuser` is launched with no `--config`, and `~/.jupyter` is first on the config search path (`jupyter_server` 2.14.1). It does not touch the culler and does not keep the server alive; it only removes the manual restart on return. It is left uninstalled pending the PI's decision, and is untested—the hook can only be exercised by a real server restart, and a parse error in that file would prevent the server from starting at all.
- **Times are displayed in IST** (`Asia/Kolkata`, +05:30, no DST) in `phase3_status.sh`, `phase3_autostart.sh` and `phase3_report.py`, at the PI's request. **Stored timestamps remain UTC**—`train.py` writes `completed_at_utc`, and a recorded time should not change meaning with the reader; only the display converts. Verified on the real record rather than a synthetic one: `2026-08-11T09:13:03+00:00` renders as `2026-08-11 14:43:03 IST`. `run_phase3.sh` and `phase3_guard.sh` still log UTC and were left alone deliberately: both were executing, and bash reads a script incrementally as it runs, so editing one can corrupt its parse mid-run.
- **Wall clock is measured per attempt, not per seed.** `train.py` times from the start of the attempt that reached the final step, so an interrupted seed under-reports its true cost. Seed 0 was interrupted twice and records 6957 s, which is its final attempt only. `phase3_report.py` now prints this caveat beside the figure. No result depends on wall clock.

2026-08-10

First session on the L40S box. Environment built, pipeline reproduced from a clean tree, custom kernel executed for the first time. No model trained.

- **Environment.** `torch 2.6.0+cu124`, `triton 3.2.0`, Python 3.10, venv `3d-gen/` (added to `.gitignore`). `torch.cuda.device_count() = 2`; both devices report NVIDIA L40S, compute capability 8.9, 44.4 GiB.
- **NVML is broken on this box and `nvidia-smi` does not run.** The loaded kernel module is 580.126.09 while the userspace libraries are 580.173.02. CUDA is unaffected—`cuInit` returns 0, both devices enumerate and compute—but NVML-based utilisation and temperature logging is unavailable until the modules are reloaded, which needs root. Recorded because the run config is supposed to carry the driver version, and the number `nvidia-smi`

would have reported cannot be obtained here.

- **Pipeline reproduced exactly from a clean tree.** `phase1_acquire.py` assembled **7,108,483** unique upper-triangle band entries with all three auxiliary md5s verified; `phase1_features.py` reproduced insulation $r = +0.9969$ ($n = 21{,}740$) and compartments $r = +0.9759$ ($n = 22{,}250$) against 4DN's own tracks, with 21,519/27,679 bins carrying a complete ϕ; `phase1_dataset.py` built **5,422 / 273 / 242** train/val/test windows with drop counts 1,054 (N fraction), 799 (structural coverage), 126 (buffer), 2 (short). Every one of these matches `docs/data_card.md` exactly, so the data layer is confirmed reproducible on a second machine rather than only on the machine that built it.
- `data/pilot_manifest.json` **now differs from the committed copy, and none of the differences are content.** The committed manifest was written on Windows. Rebuilding on Linux changed (i) path separators, `data\raw\` to `data/raw/`; (ii) the byte size of the two text files, because the Windows copies had CRLF line endings —`chr9.fa` shrank by exactly 2,306,580 bytes against 2,306,580 lines, and the GTF by exactly 169,673 bytes against 169,673 lines, one byte per line in each case; and (iii) the md5 of the two `.npz` files, whose byte sizes are nevertheless *identical*, because a zip central directory records the creating platform—`create_system` is 3 (Unix) here and 0 (Windows) there. The md5s of the three files downloaded from 4DN, which are the ones checked against the 4DN API, are unchanged. Band contents re-verified directly: 7,108,483 entries, 0 non-finite, min 4.25×10^{-6} / median 3.10×10^{-4} / max 0.942, 27,679 bins, 5,345 (19.31%) without a balancing weight—every figure matching `docs/data_card.md` §4. Whether to commit the Linux manifest is a judgement call left to the PI; a manifest whose md5s are platform-dependent cannot serve as a cross-platform integrity check for the derived files, only for the downloaded ones.
- `test_model.py`**: 16/16 passed** on this box, unchanged.
- **Triton kernel executed for the first time and validated: 34/34 checks passed** (`validate_kernel.py`, exit 0) across no-p, with-p, multi-chunk and ragged-final-chunk configurations, including the assertion that $\partial L/\partial p \neq 0$.
- **Three kernel defects fixed, all in the Triton lowering rather than the mathematics.** The hand-derived adjoint was correct as written. (i) `CHUNK` was a module-level global, which Triton refuses to read from a `@jit` function; it is now a `tl.constexpr` kernel parameter. (ii) The chunk walk used `range(NCHUNK-1,-1,-1)`; Triton lowers `range` to `scf.for`, which requires a non-negative step, so both reversed loops now count up and invert the index by hand. (iii) The inner loops used `tl.minimum(CHUNK, L-t0)` as a trip count; they now always run `CHUNK` times and predicate on $t < L$.
- **The backward is not bitwise reproducible.** `dBmat`, `dCmat` and `dp` are accumulated with `tl.atomic_add` across the d_{inner} programs, so their summation order varies between runs. Measured relative spread over five repeats at $B = 2$, $D = 512$, $L = 512$: 1.0×10^{-6}, 9.6×10^{-7} and 8.9×10^{-7} respectively; `du`, `ddelta`, `dA` and `dDskip` were bitwise identical. This is fp32 rounding noise, not a defect, but it means seed-identical runs reproduce to fp32 tolerance and not to the bit. The paper must not claim bit-exact reproducibility.
- **Kernel checked beyond the supplied harness**, which runs at $D = 64$. Added checks at the real width $d_{\text{inner}} = 512$, at 64 chunks, and at $L = 1$ and $L = 65$; all agree with the reference. Width matters because the cross-channel atomics only contend at width.
- **Scan checkpoint memory measured**, since it bounds the Phase 3 batch size: 16.0 MiB per sample per scan at $d_{\text{inner}} = 512$, $d_{\text{state}} = 16$, $L = 32{,}768$, which is 0.50 GiB per sample across 16 layers and 2 directions, or 4.0 GiB at batch 8 before any other activation.
- **TeX Live installed user-local** (TinyTeX). The box had no LaTeX engine and no root, so `build_project_doc.py` could not run at all.

2026-08-10 (second entry) — Phase 3 training harness

`scripts/train.py` written. Results are recorded in a later entry; this one covers the harness and the platform findings only.

- **NCCL cannot be used on this box.** It calls `nvmlInit_v2()` during topology discovery, which fails for the same driver mismatch that breaks `nvidia-smi`. `init_process_group("nccl")` is lazy and succeeds anyway, so the failure surfaces at the first collective inside DDP's constructor. The script therefore probes NVML directly and falls back to **gloo**. Measured cost of the fallback: none detectable—two GPUs give 2.00× the single-GPU throughput (0.756 windows/s on one L40S, 1.51 windows/s on two).
- **PyTorch's CUDA caching allocator also calls NVML** when it reports memory pressure, so an out-of-memory condition surfaces as `INTERNAL ASSERT FAILED ... nvmlInit_v2_` rather than as `CUDA out of memory`. Anyone reading that traceback later should read it as OOM.
- **Kernel made 1.42× faster, and re-validated.** The scan's state vector is $d_{\text{state}} = 16$ elements wide, so Triton's default `num_warps=4` gave each program 128 lanes for 16 lanes of work. Setting `num_warps=1` took the real config from 7.55 to 5.30 s/step. Per-step losses were identical to the default across the benchmark, and `validate_kernel.py` was re-run afterwards: 34/34.
- **Gradient checkpointing implemented and then measured not to help.** The hypothesis was that the scan is latency-bound and a bigger batch bought with recompute would be nearly free. Batch scaling turned out to be linear—the GPU is already saturated at batch 2—so checkpointing costs its recompute undiluted: 1.32 s/window at batch 2 without it against 1.49 s/window at batch 8 with it. The code is retained behind `--grad-checkpoint`, off by default, as the lever to reach for if d_{model} or the window grows.
- **fp32 throughout, no autocast**, because the scan was validated in fp32 only. Training the experiment in bf16 would put it back on an unvalidated numeric path through a hand-written recurrence.
- **The baseline runs the custom Triton scan** even though it never supplies p, so the matched-compute claim against the structural arm is real rather than nominal.
- **N is excluded from masking and from the loss.** 12.00% of chr9 is N and a window may carry up to 10%. Including N would deflate bits/nucleotide by an amount that varies with each window's N content. The reported metric is therefore bits per *resolved* nucleotide, and its uniform-random floor is exactly 2.000 bits.
- **τ is measured two ways and they are not interchangeable.** `BiMambaLM.tau_stats()` derives τ from `dt_proj.bias` alone. That is a fair proxy at initialisation, where `dt_proj.weight` is small, but after training Δ is dominated by the input-dependent term $W_{dt}\delta'_t$, and a bias-only τ can be badly wrong. The script logs `tau_stats()` because the runbook asks for it, and separately measures *empirical* τ from the Δ actually produced on validation batches. **The F4 gate must be read off the empirical numbers.**
- **`num_workers` defaults to 0.** `WindowDataset` owns an `np.random.default_rng` for reverse-complement augmentation; under multiple loader workers each worker copies it, so the augmentation stream would depend on how the sampler sharded and seeds would stop being reproducible.

2026-08-07

- **Phase 0 completed.** Eleven papers mapped. Two competitors found that were not on the original reading list: Evo2HiC, which is closer to the thesis than CHROME, and Lee's Evo 2 probe, which supports it.

- **Differentiator retired.** "Structure at training, sequence-only at inference" was dropped as a claim against Evo2HiC, whose DNA-only encoder has the same property. It still holds against CHROME.
- **Forward-citation sweep.** 318 papers checked; no work probes Akita or Orca representations on a non-folding task. The transfer gap stands.
- **Mechanism (a) selected**; (b) and (c) documented but not pursued.
- **Memory horizon measured.** $\tau_{\max} = 994$ tokens against a 5,000-token bin. Consequence: ϕ interpolation became mandatory, and a trained-τ measurement was added as a Phase 4 gate.
- **Pilot acquired and validated.** Insulation $r = +0.9969$ and compartments $r = +0.9759$ against independent 4DN tracks. A TAD and a ChIA-PET-corroborated loop confirmed visually.
- **Three assembly traps caught.** A 1,425 kb "loop" that was sparse-corner noise; an hg19 *ABL1* coordinate used to choose a GRCh38 region; and the standard GEO loop list for this dataset, which is hg19—detected because its largest chr9 anchor exceeds the length of GRCh38 chr9.
- **Dataset layer built.** Splits verified exact after a first version allowed held-out windows to straddle their region edge.
- **Model implemented; two bugs found by unit tests.** The reverse-complement sign flip was being applied to the encoded signal rather than to raw ϕ, where the encoder has already mixed all eight coordinates, making the operation meaningless. Separately, A was broadcast by trailing-axis alignment, which matches the state dimension against sequence length and only agrees when $L = d_{\text{inner}}$; the first test passed only because it used $L = 64$ with $d_{\text{inner}} = 64$.
- b_g **changed from** -8 **to** -4**.** The gradient through softplus is $\sigma(b_g)$, so -8 attenuated the gate's learning signal roughly 3000-fold—failure mode F2, self-inflicted by our own initialisation. Measured effect on the gate gradient: $2.06 \times 10^{-7} \to 7.69 \times 10^{-6}$.
- **Encoder bottleneck identified.** $\partial L/\partial s = W_{\Delta s}^{\top} \partial L/\partial \Delta$ with $W_{\Delta s}$ zero-initialised, so the encoder receives exactly zero gradient on step zero and roughly 10^{-9} afterwards against 10^{-4} for the matrix in front of it. Recommendation: remove the encoder and feed the eight standardised features directly at $d_s = 8$, costing +1.700% parameters and eliminating F8.
- **Triton kernel written** for the permeability term, with a GPU validation harness. Unverified until executed.

9 Open items

- **Decide whether bit-exact reproducibility is required.** If it is, the atomics in the backward must be replaced with a deterministic reduction, at a performance cost. If it is not—the reasonable reading, given that the Phase 5 gate is across-seed variance rather than a single number—say so explicitly, because the spread is measured and small.
- **Restore** `nvidia-smi` by reloading the NVIDIA kernel modules, or accept that no driver-level utilisation figure can be logged alongside the runs.
- **Confirm the encoder removal** ($d_s = 8$, no encoder), which changes parameter figures recorded in three documents.
- **Read CHROME line by line.** The summary here came from automated extraction of the PMC full text.
- **Obtain Orca's Results section**, paywalled at *Nature Genetics* with no PubMed Central deposit. The claim that its representations were never evaluated on a non-folding task rests on the abstract, all ten extended-data captions and the section headings, not the body.

- **Re-check Evo2HiC** for a revision adding variant evaluation before Phase 4 commits GPU time; that would close the one contribution nothing currently contests.
- **Re-check Lee's preprint** for peer-review status before it carries weight in a methods section.

10 What happens next

Phase 3 trains the sequence-only baseline, logging every hyperparameter, seed and metric before any structural model exists, so no flattering comparison can be selected after the fact. That run must also report the trained memory horizon, which decides whether a model of this size can represent TAD-scale dependencies at all. If it cannot, the mechanism is re-scoped to sub-TAD structure or the model is enlarged, and that decision is taken before Phase 4 rather than after.

Every measured number in this document traces to a named script: `phase1_acquire.py`, `phase1_features.py`, `phase1_validate_visual.py`, `phase1_dataset.py`, `param_accounting.py`, `f4_memory_horizon.py`, `test_model.py`. Literature claims are sourced in `docs/related_work.md`; the mechanism is specified in `docs/architecture_spec.md`; data provenance is in `docs/data_card.md`.